

IT628 (Winter 2024-25), Lab 4: IO redirection with descriptors

In this lab you will be creating your own mini-shell, that is, your program will read in command lines, distinguish between program name, command-line arguments, < and > for redirection, and launch new processes matching the command. It will require you to use fork and exec (hopefully that's obvious) and it will also require you to change standard input or standard output when you receive commands like "mmame < foo.txt" or "mmame > foo.txt". This we'll do at the file descriptor level. So ask yourself: Which file descriptor is "standard in", "standard out"? How do I change that file descriptor so that it refers to a connection to some other file?

Here is a link to [example exec calls](#) (and other details about the exec functions).

Getting checked off: Show your solutions to a TA.

Background

The tar archive contains code for a utility called mmame (min-max-average-median). This utility will read the input from stdin, i.e. reading until end-of-file (^D, CONTROL-D) is encountered, then printing the results:

```
$ mmame
1 2 3
^D
1.000 3.000 2.000 2.000
$
```

mmame reads from stdin
the user enters data
the user indicates end-of-input
results are printed
back to the shell prompt

The mmame utility will also read from a file named on the command line, but after the file has been processed, the program continues reading from standard input! Note that results are not printed until the user enters ^D. This allows the user to continue entering data from stdin after mmame first reads data from the named file:

```
$ mmame test_data.dat
1 2 3
^D
1.000 32767.000 16601.027 16617.000
$
```

mmame reads data from test_data.dat
the user continues to enter values
the user then indicates end-of-input
results are printed
back to the shell prompt

mmame behaves differently when the shell redirects stdin to read from a file, as follows:

```
$ mmame < test_data.dat
55.000 32767.000 16650.824 16633.000
$
```

mmame reads from stdin (redirected from test_data.dat)
results are printed
back to the shell prompt

Notice that ^D is not entered by the user when IO redirection is handled by the shell.

The command make mmame will compile-and-link the program. Do that, and experiment with it a little bit.

Part 1: Minimal minishell (0 pts)

For Part 1 you are to create a program named `minishell`. You are given skeleton code for `minishell` in a file named `minishell.c`. This is your starting point. If you give the `make` command, it'll be compiled-and-linked ... although the version you're given doesn't do anything. Read the source code and follow the numbered comments! `minishell` works as follows:

1. The `minishell` prompt is `IT628>`
2. `minishell` reads "minishell commands" from `stdin`, but it is a very simple shell: it "understands" only how to run the program named on its command line. We're testing the `minishell` with the `mmame` program, but you may NOT assume the user won't try to run a program other than `mmame`.
3. Your `minishell` should read user input, then `fork/exec` as appropriate to execute `mmame`.

Here are some examples of the `minishell` in action for a correct Part 1 solution:

```
$ minishell
IT628> mmame
30 25 60 30
^D
25.000 60.000 36.250 30.000
IT628>
```

Start the minishell
IE411 is the minishell prompt. Here we're running `mmame` program
User enters data (`mmame` reading from `stdin`)
User indicates end-of-input with the `^D` character
This is `mmame` output
`mmame` terminated: back to the minishell prompt


```
$ minishell
IT628>
IT628>
```

User enters nothing on the command line: just presses the `ENTER` key
(returns the user to the minishell prompt)


```
$ minishell
IT628> grok
IT628> minishell: grok: command not found
IT628>
```

User enters a command that can't be executed
`minishell` prints an error message
...and returns the user to the minishell prompt

Part 2: Reading direct from file (0 pts)

Add to a correctly functioning Part 1 the ability to execute `mmame` reading directly from a file. Do not attempt Part 2 unless Part 1 works completely and correctly. Here are two examples of the `minishell` in action for a correct Part 2:

```
$ minishell
IT628> mmame test_data.dat
^D
55.000 32767.000 16650.824 16633.000
IT628> mmame test_data.dat
1 2 3
^D
1.000 32767.000 16601.027 16617.000
IT628>
```

Part 3: Input Redirection (checkoff - 1 pt)

Add to a correctly functioning Part 2 the ability to execute `mname` using input redirection. Do not attempt Part 3 unless Part 2 works completely and correctly. Here's an example of the minishell in action for a correct Part 3:

```
IT628> mname < test_data.dat
55.000 32767.000 16650.824 16633.000
```

You may assume that only well-formed commands are entered on your minishell command line. A well-formed command looks like this:

```
mname < filename
```

You DO NOT have to worry about handling lines like these:

```
mname foo.dat <    (< In The Wrong Location)
mname <foo.dat      (Missing Space After <)
mname <             (Missing Filename After <)
< mname foo.dat     (mname not first)
```

Part 4: Output Redirection (checkoff – 1 pt)

Add to a correctly functioning Part 2 the ability to execute `mname` using output redirection. Do not attempt Part 4 unless Part 2 works completely and correctly. If the file to which output is redirected already exists, the old version will be overwritten. Here's an example of the minishell in action for a correct Part 4:

```
IT628> mname > foo.dat
1 2 3
^D
IT628> ^D
$
```

You may assume that only well-formed commands are entered on your minishell command line. A well-formed command looks like this:

```
mname > filename
```

You DO NOT have to worry about handling lines like these:

```
mname filename >   (> In The Wrong Location)
mname >filename     (Missing Space After >)
mname >             (Missing Filename After >)
> mname filename    (mname not first)
```